

01.02.26



Lab 3

Gervigreind

Sindri Rafn

Númi Jökull

Task 1: What constitutes a state of the environment? Which information about the environment is not in the state and why?

In the environment.py file the state is defined as:

```
State = collections.namedtuple('State',['position','orientation','turned_on','dirts'])
```

Then in the Environment class, we see that it is initialized with these values:

```
width = 0  
height = 0  
home = (0,0)  
dirts = ()  
obstacles = set()  
current_state = None
```

Now a state of the environment only stores information about the environment that is subject to change whilst the agent is acting, the robot's current position, the direction it is facing, whether it is turned on or off, and which dirt locations are still left to clean. This is the information we use to assess the next steps of the agent and to keep track of whether the agent has achieved its goal or not.

Static information like the board size, obstacles, and home position is kept in the environment rather than the state because it does not change and does not need to be stored in every state.

Task 2: Assuming the environment has width W , length L , D dirty spots and relatively few obstacles compared to accessible spaces.

Assess the following search algorithms wrt. their completeness, optimality, space and time complexity in the given environment: Depth-First Search, Breadth-First Search, Uniform-Cost Search, A* Search. If one of the algorithms is not complete, how could you fix it?

State space:

Positions: $W * H$

Orientation: 4 $\rightarrow$ N/W/S/E

Turned_on: 2 $\rightarrow$ on/off

Dirts: 2^d $\rightarrow$ because a dirt cell is either remaining or has been sucked up.

$$N \leq (W*H)*4*2*2^d = 8WH * 2^d$$

Branching factor:

The branching factor when the agent is turned off is $b=1$ because the only legal action is turn_on.

When the agent is on the branching factor is at most 5, (TURN_RIGHT, TURN_LEFT, GO, SUCK, TURN_OFF)

DFS:

Complete: Not always, under certain circumstances it can loop forever.

Optimal: No, it depends on what path reaches the goal first

Time: $O(N)$

Memory: $O(N)$

BFS:

Complete: Yes, because the board is finite

Optimal: only if all actions have the same cost

Time: $O(N)$

Memory: $O(N)$

UCS:

Complete: Yes, all actions have a positive cost ≥ 1

Optimal: Yes, it always finds the cheapest solution

Time: $O(N \log N)$

Memory: $O(N)$

A* search:

Complete: yes, all actions have a positive cost ≥ 1

Optimal: Yes, (depends if the heuristic is admissible)

Time: $O(N \log N)$

Memory: $O(N)$

Task 3: The code contains a heuristic function that estimates the number of steps needed to finish the task from a given state. Is the heuristic admissible? Is it consistent? Explain.

What makes a heuristic function admissible? A heuristic function is admissible if it *never* overestimates the true minimal cost to reach a goal. This heuristic function uses manhattan distances (taxicab distance) which measures the distance between 2 points in a grid based system just like our boards are set up. Since the robot can only move one step at a time and obstacles can only make the path longer, this distance is always a lower bound. The heuristic also counts required actions like cleaning dirt and turning the robot off, so it does not overestimate the remaining cost and is therefore admissible.

Task 5: Run your search on all the given environments and record the results. Report on the results and compare with your estimated complexity.

Here are the results we got:

vacuumcleaner_obstacles_1:

environment is of size 5 x 5

planning took 0.02s

number of node expansions: 1925

node expansions per second: 103528.0

maximal frontier size: 207 nodes

plan length: 42

effective branching factor: 2.02

plan cost: 42

NFO(18:20:04.810): Game over! results: 63

INFO(18:20:04.811): runtimes (in ms): 42

vacuumcleaner_obstacles_2:

environment is of size 5 x 5

planning took 0.02s

number of node expansions: 1718

node expansions per second: 111111.1

maximal frontier size: 179 nodes

plan length: 46

effective branching factor: 1.95

plan cost: 46

INFO(18:19:42.633): Game over! results: 59

INFO(18:19:42.634): runtimes (in ms): 57

vacuumcleaner_obstacles_3:

environment is of size 20 x 40

planning took 16.18s

number of node expansions: 1813160

node expansions per second: 112094.6

maximal frontier size: 29544 nodes

plan length: 286

effective branching factor: 2.55

plan cost: 286

INFO(18:18:36.005): Game over! results: 724

INFO(18:18:36.007): runtimes (in ms): 16137

vacuumcleaner_obstacles_4:

environment is of size 20 x 40

planning took 7.78s

number of node expansions: 922718

node expansions per second: 118559.2

maximal frontier size: 43153 nodes

plan length: 202
effective branching factor: 2.59
plan cost: 202
INFO(18:17:47.774): Game over! results: 808
INFO(18:17:47.774): runtimes (in ms): 7792

vacuumcleaner_obstacles_5:

environment is of size 20 x 40
planning took 10.18s
number of node expansions: 1216710
node expansions per second: 119572.5
maximal frontier size: 52072 nodes
plan length: 170
effective branching factor: 2.73
plan cost: 170
INFO(18:16:48.271): Game over! results: 840
INFO(18:16:48.271): runtimes (in ms): 10528

vacuumcleaner_obstacles_7:

environment is of size 20 x 40
planning took 0.70s
number of node expansions: 105184
node expansions per second: 151214.4
maximal frontier size: 2304 nodes
plan length: 222
effective branching factor: 2.14
plan cost: 222
INFO(18:22:20.289): Game over! results: 724
INFO(18:22:20.289): runtimes (in ms): 704

vacuumcleaner_obstacles_8:

environment is of size 20 x 40
planning took 1.19s
number of node expansions: 221240
node expansions per second: 185439.0
maximal frontier size: 3920 nodes
plan length: 259
effective branching factor: 2.21
plan cost: 259
INFO(18:23:08.629): Game over! results: 703
INFO(18:23:08.629): runtimes (in ms): 1356

vacuumcleaner_obstacles_9:

environment is of size 20 x 40
planning took 2.93s
number of node expansions: 447157
node expansions per second: 152821.5
maximal frontier size: 7254 nodes

plan length: 270
effective branching factor: 2.32
plan cost: 270
INFO(18:23:46.834): Game over! results: 708
INFO(18:23:46.834): runtimes (in ms): 3315

vacuumcleaner_obstacles_10:

environment is of size 20 x 40
planning took 6.63s
number of node expansions: 877333
node expansions per second: 132373.6
maximal frontier size: 14978 nodes
plan length: 271
effective branching factor: 2.44
plan cost: 271
INFO(18:24:29.702): Game over! results: 723
INFO(18:24:29.702): runtimes (in ms): 7014

What we can see with these results is that the estimated complexity increases with the grid size and the number of dirt locations. In the smaller boards i.e. the 5x5 boards the search starts and finished very quickly, with fewer than 2.000 node expansions. However in the larger boards of sizes 20x40, the search expands >100.000 or sometimes > 1.000.000 nodes, and takes several seconds. The trend then matches the estimated complexity which is:

$$O(W * H * 2^D)$$

W=Width

H=Height

D=num of dirt locations